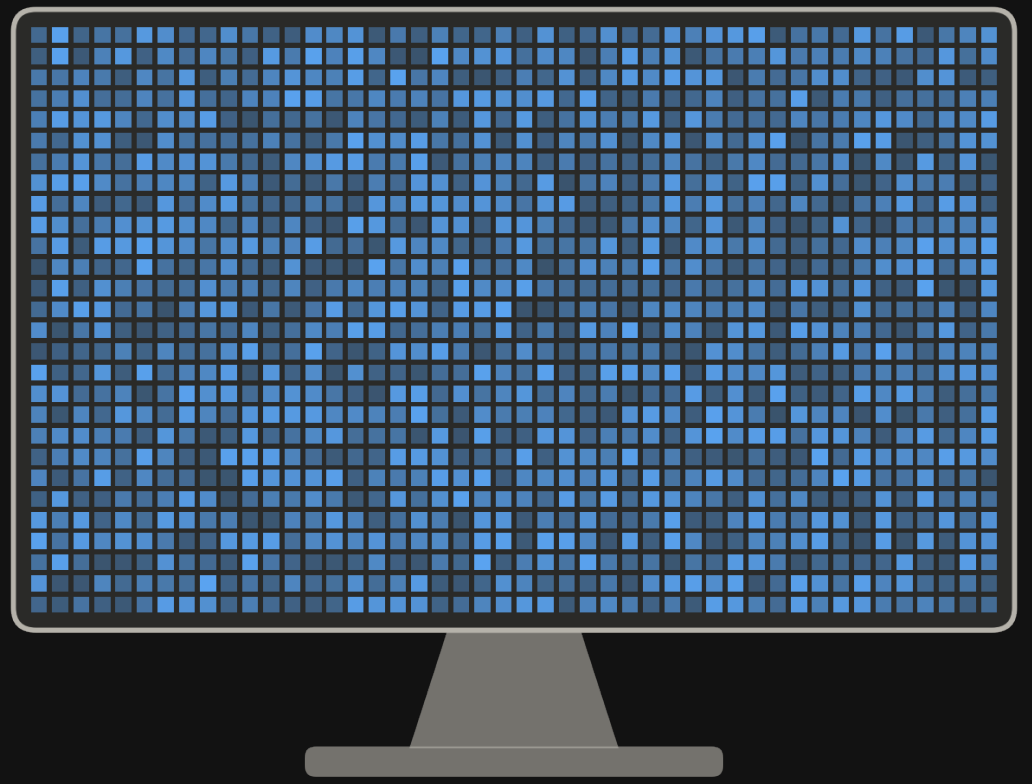
Saving the Tokens

Your agent pays again for everything it already sent.

Find the big thing. Stop sending it.



Shyamji Tiwari

Applied Scientist, Amazon

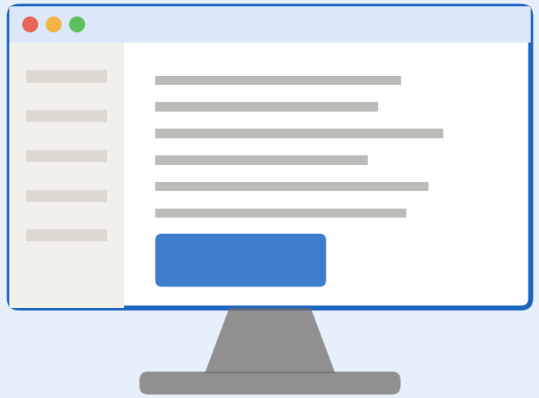
IEEE AgenticAI Summit 2026 · Bangalore

Views are my own.

Which is the bigger part of the bill?

A 20-turn session with an AI agent that reads your screen

A



The screenshots

a new one every turn (~1,200 tokens)

or

B



The system prompt

the same instructions every turn (400 tokens)

Agent = an AI that reads your screen and acts. **Turn** = one message to it, and its reply. **Token** = the unit you are billed in.

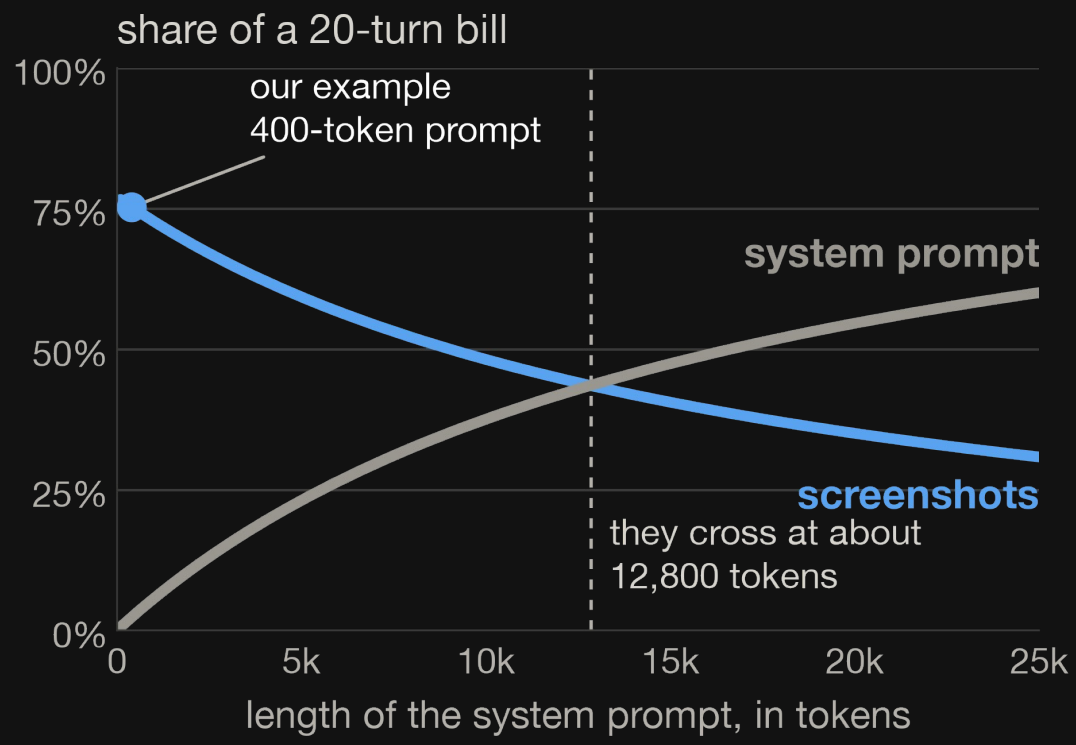
EXAMPLE MODEL

75%

of the bill is the screenshots.

System prompt (400 tokens): **2.3%**

A new screenshot every turn, and each one is **sent again.**



A long prompt can win: it never grows, but per turn it can cost more than one screenshot. Break-even prompt is about 610 × (turns + 1) tokens: 6,700 at 10 turns, 12,800 at 20, 25,000 at 40.

An example agent, not a measurement. 20 turns. Each turn: 8 s of speech (256 tokens, Gemini's published 32 a second), one 1,222-token screenshot (modelled), a 120-token reply. 400-token prompt, no caching. Change a setting and the percentages move.

numbers come from a model you can run and edit: ex3_budget.py

A token is the unit of the bill

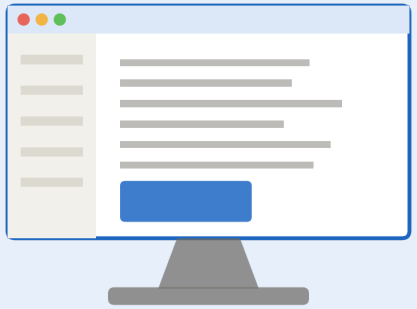
You pay for everything going **in**, not just what comes out.



≈ 750

tokens · a dense page of text

rule of thumb: 1 token is about 4 characters, a page about 3,000



1,105–2,691

tokens · one 1920×1080 screenshot

published rules of OpenAI and Claude (next slides)



256

tokens · 8 seconds of speech

Gemini: 32 tokens a second, published (others differ)

Every call sends the whole history again

Pseudo-code of most agents. The usage field is where the bill is read.

```
history = [system_prompt]
while not done:
    history.append(observe())   # screenshot, speech, tool output
    reply = model(history)      # billed for ALL of history
    log(reply.usage)            # input/output tokens = the bill
    history.append(reply)
```

Orange = what you pay for again. **Blue** = where you read what it cost.

Orange is full price unless you cache it (slide 14). The model can only read so many tokens per call: that limit is the **context window**.

How a picture becomes tokens

A modelled tiler (InternVL-style). Real vendors work differently: slide 8.

STEP 1

Your screenshot



1920×1080 pixels

STEP 2

Cut into tiles



a 4×2 grid of squares

STEP 3

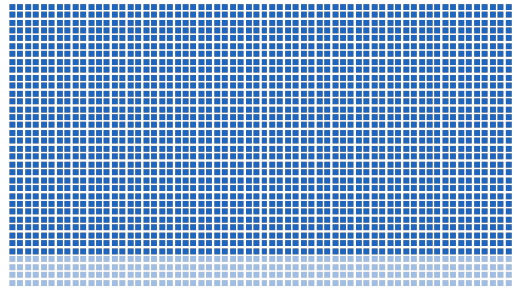
256 tokens per tile



8 tiles, plus 1 small copy of the whole image

STEP 4

2,304 tokens



one square = one token

9 tiles × 256 tokens = **2,304 tokens**

Three ways a model can turn a picture into tokens

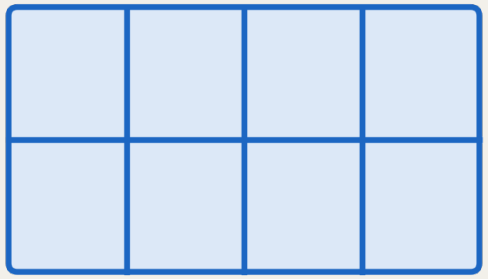
Three modelled strategies, all priced on the same 1920×1080 screenshot



Squash it

Same cost, any size

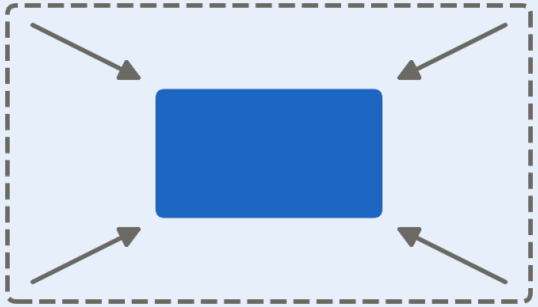
576
tokens



Cut into tiles

Cost follows the shape

2,304
tokens



Shrink to a budget

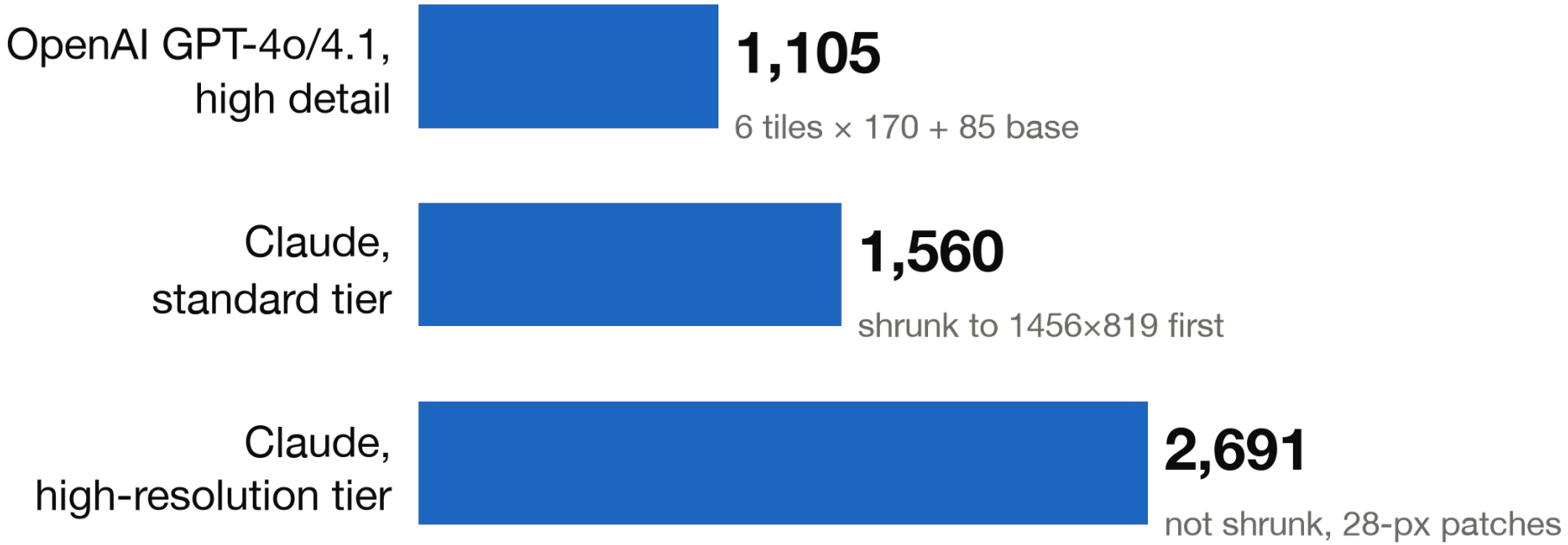
The cap is a setting

1,222
tokens

the example agent uses “shrink to a budget”, capped at 1,280

Real products price it differently

A 1920×1080 screenshot, priced by each vendor's published rule



Claude standard: a 4K screen costs the same **1,560**. The cap sets the bill.

Claude 4.7+ is the high-res tier: an upgrade can move it from 1,560 to 2,691.

Open the notebook. Run two cells.

Scan: opens in Colab
Google account + wifi

1 Press Play on
Run this first

2 Press Play on
Check the setup

3 Look for **PASS**

```
$ python3 check_setup.py

7 checks ... ok
PASS. You are ready.
```

Have Python? Use a terminal. works offline

Get the folder: **github.com/ieee-ai/agentic-ai-26**

1 Mac: ⌘-Space, type Terminal.
Windows: Start, type PowerShell.

2 cd, a space, drag the folder in, Enter.

Windows: py, not python3

Nothing to install. No API key.

No Google account? Pair with a neighbour.

EXERCISE 1

What does one image cost?

```
$ python3 ex1_image.py --size 1920x1080
modelled strategies
Squash to a square      576
Tiles (up to 12)      2,304
Tiles (up to 6)         768
Budget 1,280          1,222
Budget 256              252
real products (published rules)
Claude standard       1,560
Claude hi-res         2,691
OpenAI GPT-4o         1,105
```

Same picture: **1,105** to **2,691** tokens on real products.

The modelled strategies span 252 to 2,304 (252 is a 588×336 view).

TRY IT · notebook section 2

1. Your own screen size

   Mac: menu > About This Mac. Windows: Settings > Display.

2. Which is cheapest?

3. Double the width

4. Try 3840x2160

Four times the pixels. Same bill.

In the modelled “cut into tiles” strategy

1280×800

1.0 million pixels

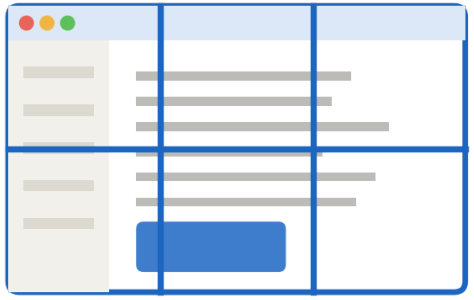


1,792 tokens

3×2 grid

1920×1200

2.3 million pixels

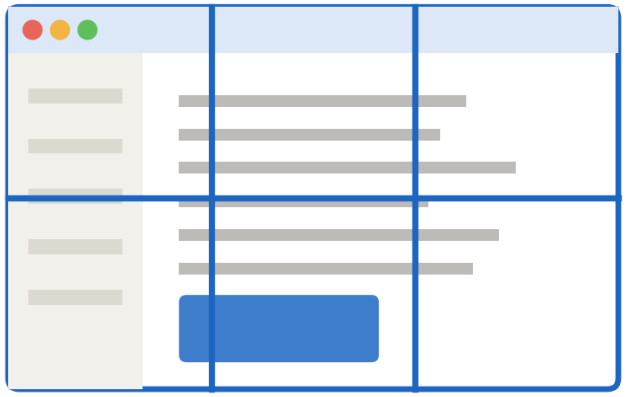


1,792 tokens

3×2 grid

2560×1600

4.1 million pixels



1,792 tokens

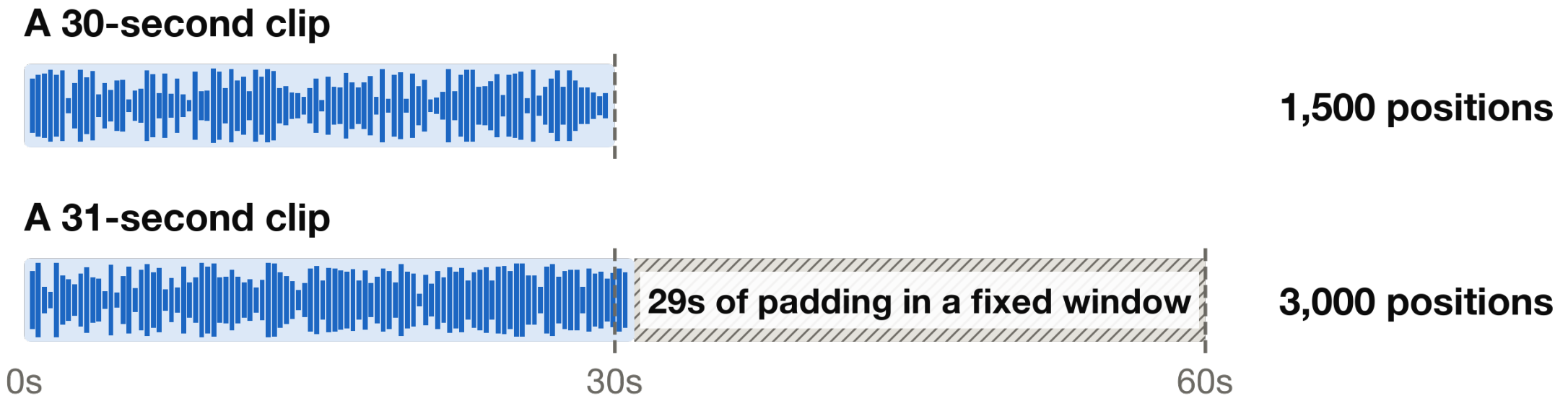
3×2 grid

A tiler prices the shape of your image, not its size.

Real rules on the same two screens: OpenAI 1,105 and 1,105, Claude standard 1,334 and 1,550, Claude hi-res 1,334 and 4,698. Measure at the sizes you ship.

Check whether your audio is padded

A Whisper-style encoder works in 30-second windows. Does your product bill for that?



33 tokens

Send the words, not the sound.

8 seconds as text: 8× fewer tokens than speech at Gemini's 32 a second (up to 45× against a padded window). Speech recognition is not priced.

window encoder: modelled · 32 tokens a second: Gemini, published

EXAMPLE MODEL

Every turn re-sends the whole conversation

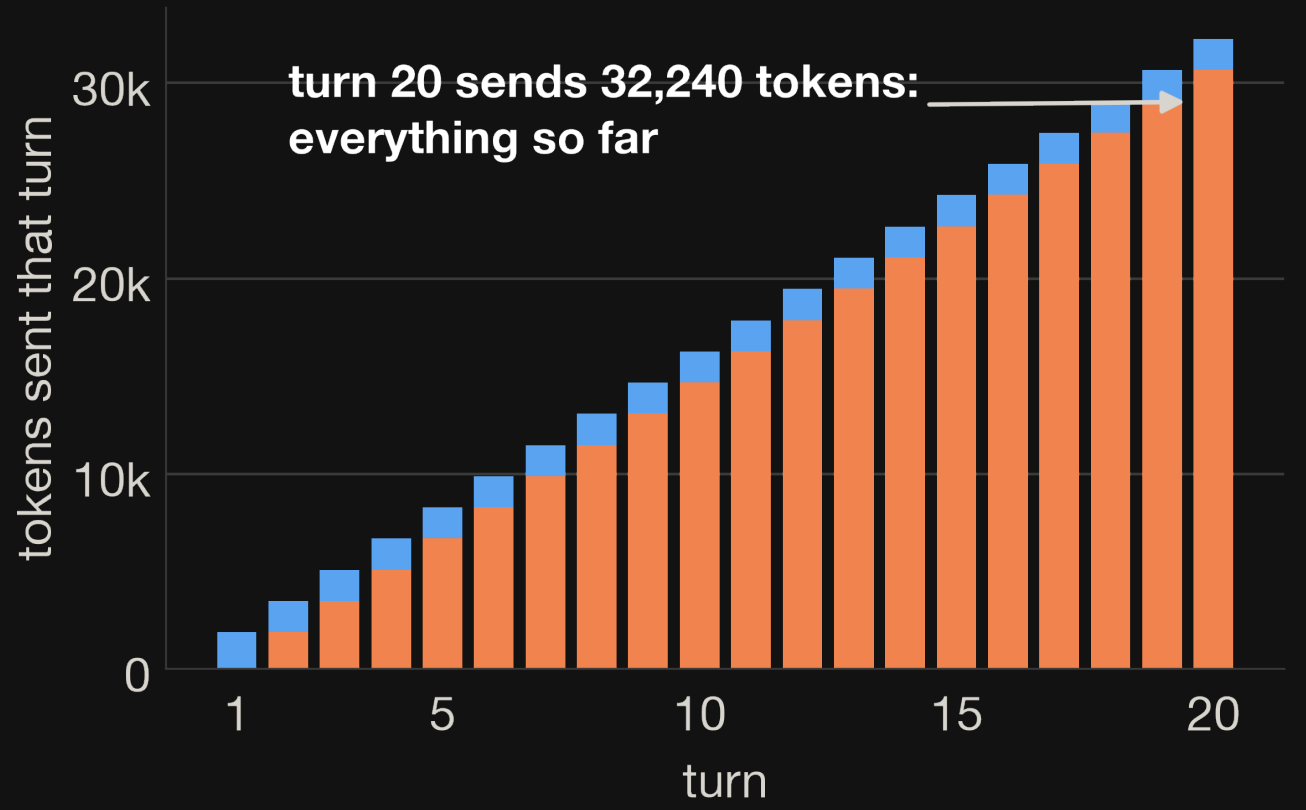
91%

of the input tokens were sent before
(20 turns, no caching)

New content **32,240**

Sent again **308,940**

Input in total **341,180**



Two things grow. The bill: double the turns, about 4× the tokens. And the request itself: this agent reaches the 128,000-token context window at turn 80. max_tokens only caps one reply.

input tokens only (output is priced separately)

EXAMPLE MODEL

Ten ways to cut the bill. Which wins?

For a voice agent that watches the screen, 20 turns

1. Cut the system prompt by 60%
2. Drop old screenshots, keep the newest
3. Halve every agent reply
4. Halve the tokens per second of speech
5. Keep only the last 5 turns
6. Pay 10% for repeated text (prefix caching)
7. Screenshot every 4th turn
8. Send a transcript, not the audio
9. Send the screen's text, not a picture (~300 tokens)
10. Shrink the image budget

ON PAPER, NOW

Most?

Least?

90 seconds. Then run:

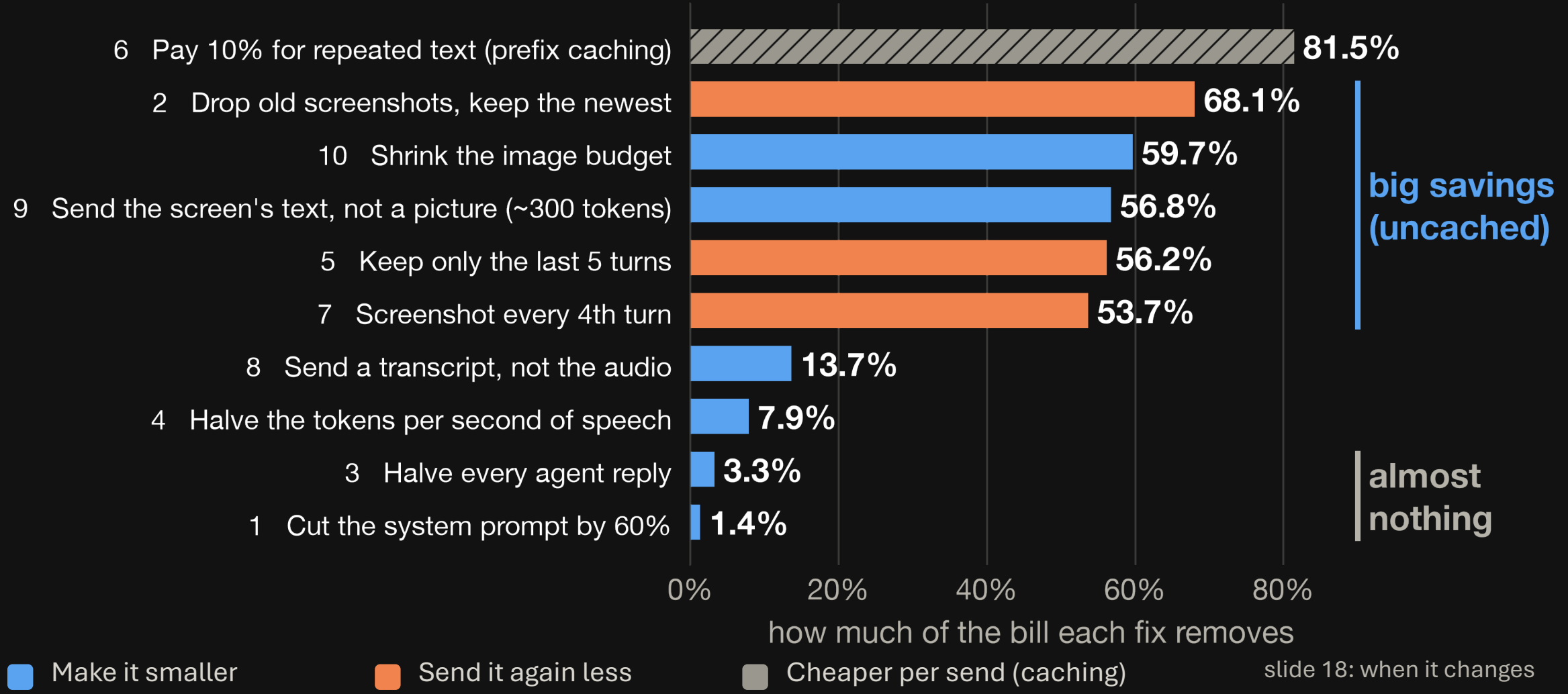
Play “Reveal”

Prefix caching: the provider remembers the start of your prompt and charges about 10% to read it again.

EXAMPLE MODEL

Ranked by what it saves, in our example agent

400-token prompt, 20 turns, uncached; ranked by tokens saved, not by what survives your task. At 4,000 tokens the prompt fix saves 12%.



Your bill is a multiplication

how **big** each thing is × how many times you **send it** × what each send **costs**

Shrinking acts on the first, pruning (dropping old items) on the second, caching on the third.

Share of the input tokens, one 20-turn session. The ranking you just saw follows these shares.

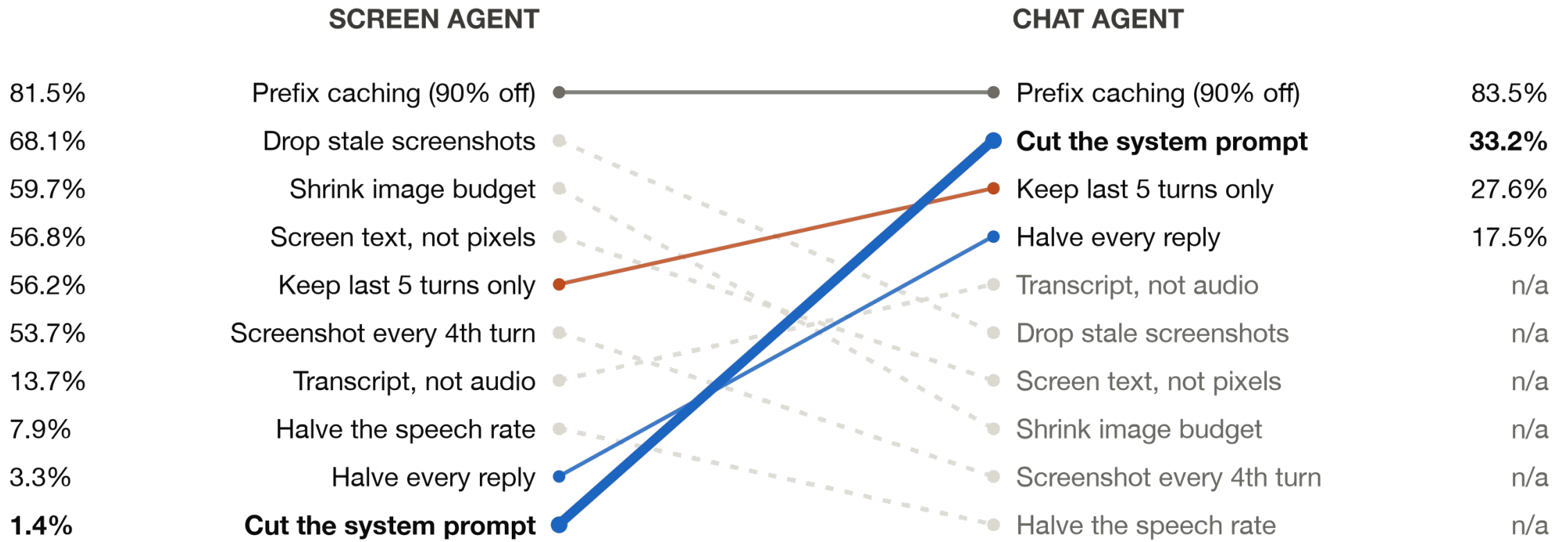


About **$1.02** a session and about **$307,000** a month at 10,000 sessions a day, 30 days. Assumed $3.00 per million input tokens.

EXAMPLE MODEL

No pictures: now the prompt is the big item

Chat agent bill: prompt 55%, replies 35%, messages 10%. Six of the ten fixes do not apply.



Control: the screen agent with the same 6,000-token prompt ranks the prompt fix only 6 of 9 design fixes (16%). It takes the pictures being gone, not just a long prompt.

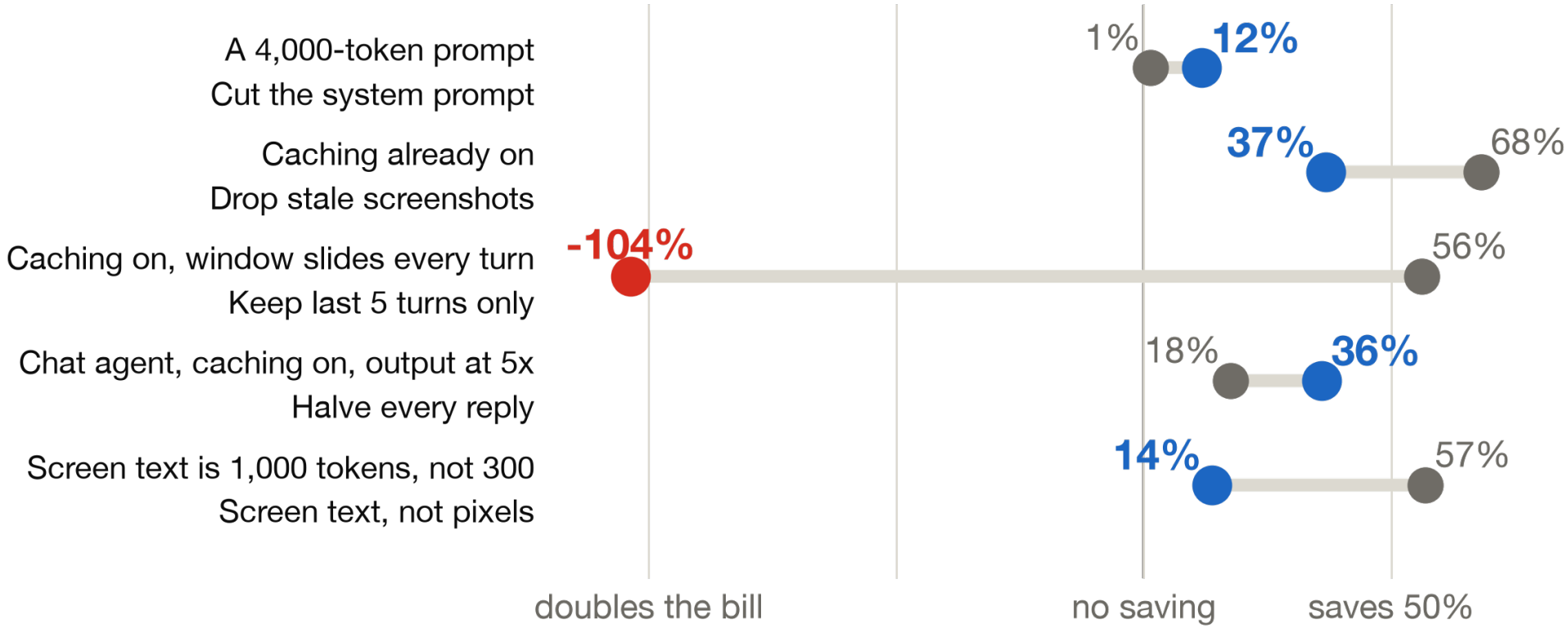
Press Play on “Two agents side by side”

```
terminal: python3 ex3_budget.py --agent both
```


Simulated 20-turn session · illustrative settings · input tokens only

That ranking is not a law

Change one assumption of the agent and a fix moves a long way



Not modelled: tool results and schemas · cache write fees and expiry · output tokens in the bill · accuracy

TAKE THIS WITH YOU

Monday morning: find YOUR big number

1

Log input tokens on every model call.

2

Run `bill_report.py` on the log.

3

See what dominates: the prefix, or what each turn adds.

4

Pick the fix aimed at that one.

Your bill = size × sends × price per send

The best fix changes with:

caching · session length · prompt size · pictures

So measure yours first.

Thank you

Shyamji Tiwari
Applied Scientist, Amazon

github.com/ieee-ai/agentic-ai-26/Saving-the-Tokens

Simulated 20-turn session · illustrative settings · input tokens only